HIGH PEFORMANCE COMPUTING

ASSIGNMENT-4

Name : B.shashank

Roll Number : 2303A51431

Batch : 21

Assignment 1: Parallel Vector Addition Scenario Add two large vectors in parallel and compare serial and parallel execution time. Objective To understand basic OpenMP parallel loop execution. Tasks

1. Initialize two large vectors.
1. Implement vector addition using a serial loop.
2. Implement vector addition using a parallel loop.
3. Compare the execution time of serial and parallel implementations. Learning Outcomes Differentiate between serial and parallel program execution. Implement OpenMP-style parallel loops using Python.

 Measure and compare execution time of serial and parallel programs. Analyze the performance benefits of parallel computing.

```
import time
import numpy as np
from concurrent.futures import ThreadPoolExecutor

# Vector size
N = 10_000_000

# Initialize vectors
A = np.random.rand(N)
B = np.random.rand(N)
C = np.zeros(N)

# -------------------------- SERIAL EXECUTION --------------------------
start = time.time()
for i in range(N):
    C[i] = A[i] + B[i]
end = time.time()
print("Serial Time:", end - start)

# -------------------------- PARALLEL EXECUTION --------------------------
def add_chunk(start, end):
    for i in range(start, end):
        C[i] = A[i] + B[i]

num_threads = 4
```

```
chunk_size = N // num_threads

start = time.time()
with ThreadPoolExecutor(max_workers=num_threads) as executor:
    for i in range(num_threads):
        executor.submit(
            add_chunk,
            i * chunk_size,
            N if i == num_threads - 1 else (i + 1) * chunk_size
        )
end = time.time()
print("Parallel Time:", end - start)
```

```
Serial Time: 8.984385251998901
Parallel Time: 3.983694076538086
```

Assignment 2: Static Scheduling in Parallel Loop Scenario Perform a vector computation using static scheduling. Objective To study static scheduling behavior in OpenMP-style parallel loops. Tasks

1. Initialize input arrays.
1. Perform vector computation using a parallel loop.
2. Measure execution time.
3. Observe work distribution among threads. Learning Outcomes Understand the concept of loop scheduling in OpenMP. Explain static scheduling and its work distribution mechanism. Implement static scheduling in parallel loops. Evaluate the effect of static scheduling on execution time.

```
import numpy as np
import time
from concurrent.futures import ThreadPoolExecutor

N = 1_000_000
A = np.random.rand(N)
B = np.zeros(N)

num_threads = 4
chunk_size = N // num_threads

def compute(start, end, tid):
    print(f"Thread {tid} handles range {start} to {end}")
    for i in range(start, end):
        B[i] = A[i] * A[i]

start_time = time.time()
with ThreadPoolExecutor(max_workers=num_threads) as executor:
    for tid in range(num_threads):
        s = tid * chunk_size
        e = N if tid == num_threads - 1 else (tid + 1) * chunk_size
        executor.submit(compute, s, e, tid)
```

```
end_time = time.time()
print("Execution Time:", end_time - start_time)
```

```
Thread 0 handles range 0 to 250000Thread 1 handles range 250000 to 500000
Thread 2 handles range 500000 to 750000
Thread 3 handles range 750000 to 1000000

Execution Time: 0.4210777282714844
```

Assignment 3: Load Imbalance in Parallel Execution Scenario Analyze parallel loop performance when different iterations perform different amounts of work. Objective To understand load imbalance and its impact on scheduling. Tasks

1. Create an array of random numbers.
1. Perform light computation for some iterations.
2. Perform heavy computation for other iterations.
3. Measure and analyze execution time. Learning Outcomes Identify load imbalance in parallel programs. Understand how uneven workload affects performance. Analyze the limitations of static scheduling for imbalanced workloads. Relate load imbalance issues to scheduling strategies in OpenMP.

```
import random
import time
from concurrent.futures import ThreadPoolExecutor

N = 100000
data = [random.randint(1, 100) for _ in range(N)]

def work(start, end, tid):
    for i in range(start, end):
        if data[i] < 50:
            sum(j*j for j in range(100))    # light work
        else:
            sum(j*j for j in range(5000))  # heavy work
    print(f"Thread {tid} completed")

num_threads = 4
chunk = N // num_threads

start = time.time()
with ThreadPoolExecutor(max_workers=num_threads) as executor:
    for tid in range(num_threads):
        executor.submit(
            work,
            tid * chunk,
            N if tid == num_threads - 1 else (tid + 1) * chunk,
            tid
        )
end = time.time()
```

```
print("Total Time:", end - start)
```

```
Thread 3 completed
Thread 2 completed
Thread 0 completed
Thread 1 completed
Total Time: 22.646467208862305
```

Assignment 4: Parallel Reduction (Synchronization)

29-01-26 Thursday Scenario Compute the sum of all elements in an array using parallel execution. Objective To eliminate race conditions using reduction. Tasks

1. Initialize a large array.
1. Attempt to sum elements in parallel.
2. Use OpenMP-style reduction to avoid race conditions.
3. Display the final sum. Learning Outcomes

 Identify race conditions in parallel programs. Understand the need for synchronization in shared data access. Apply reduction techniques to safely accumulate shared variables. Implement correct and race-free parallel reductions.

```
import numpy as np
from concurrent.futures import ThreadPoolExecutor

arr = np.random.rand(1_000_000)
num_threads = 4
chunk = len(arr) // num_threads

def partial_sum(start, end):
    return sum(arr[start:end])

results = []
with ThreadPoolExecutor(max_workers=num_threads) as executor:
    for i in range(num_threads):
        s = i * chunk
        e = len(arr) if i == num_threads - 1 else (i + 1) * chunk
        results.append(executor.submit(partial_sum, s, e))

final_sum = sum(r.result() for r in results)
print("Final Sum:", final_sum)
```

```
Final Sum: 499817.2489515851
```

Assignment 5: Barrier Synchronization (Two-Phase Computation) Scenario Perform a computation in two phases ensuring correct synchronization between phases. Objective To demonstrate barrier synchronization in parallel execution. Tasks

1. Perform computation in Phase 1.

1. Synchronize all threads.
2. Perform computation in Phase 2.
3. Display completion message. Learning Outcomes

 Understand barrier synchronization in parallel computing. Implement multi-phase parallel programs. Ensure correct execution order using synchronization techniques. Explain the role of barriers in OpenMP-style parallel execution.

```python
from threading import Barrier, Thread
import time

barrier = Barrier(4)

def task(tid):
    print(f"Thread {tid} Phase 1 started")
    time.sleep(1)
    print(f"Thread {tid} Phase 1 finished")

    barrier.wait()   # Barrier synchronization

    print(f"Thread {tid} Phase 2 started")
    time.sleep(1)
    print(f"Thread {tid} Phase 2 finished")

threads = []
for i in range(4):
    t = Thread(target=task, args=(i,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("All threads completed both phases")
```

```
Thread 0 Phase 1 started
Thread 1 Phase 1 started
Thread 2 Phase 1 started
Thread 3 Phase 1 started
Thread 0 Phase 1 finished
Thread 1 Phase 1 finished
Thread 2 Phase 1 finished
Thread 3 Phase 1 finished
Thread 3 Phase 2 started
Thread 0 Phase 2 started
Thread 1 Phase 2 started
Thread 2 Phase 2 started
Thread 0 Phase 2 finishedThread 3 Phase 2 finished
Thread 1 Phase 2 finished
```

```
Thread 2 Phase 2 finished

All threads completed both phases
```